Lab II

Basic Syntax, In/Output, Data Types & Arithmetic Operators in C++

Asad Ullah

Electrical Ai

Semester 2nd

Section B

Reg No: BF25NWELE0686

Learning objectives:

Basic syntax of C++

• Input and output using cin and cout

• Different data types

• Memory size and address of variables

• Arithmetic operations

• Temperature conversion

Theory:

➢ Basic Structure of C++ Program:

Every C++ program consists of:

• Header file (#include)

• Namespace (using namespace std;)

• Main function (int main()) • Program statements inside curly braces { }

Example:

```
#include <iostream>
using namespace std;

int main()
{
   cout << "Hello World";
  return 0;
}
```

Input and Output in C++:

C++ uses:

- **cout** → to display output
- **cin** → to take input from user
- << → insertion operator
- >> → extraction operator
-

Example:

```
#include <iostream>
using namespace std;

int main()
{     int num;

   cout << "Enter a number: ";     cin >> num;

   cout << "You entered: " << num;

  return 0;
}
```

Input (Example)

Enter a number: 5

Output

You entered: 5

Data types:

Type	Keyword
Boolean	Bool
Character	Char
Integer	Int
Floating point	float
Double floating point	double
Valueless	Void
Wide character	wchar_t

- Several of the basic types can be modified using one or more of these type modifiers:

- signed
- unsigned
- short
- long

- The following table shows the variable type, how much memory it takes to store the value in memory, and what is maximum and minimum value which can be stored in such type of variables.

Type	Typical Bit Width	Typical Range
char	1byte	-127 to 127 or 0 to 255
unsigned char	1byte	0 to 255
signed char	1byte	-127 to 127
int	4bytes	-2147483648 to 2147483647
unsigned int	4bytes	0 to 4294967295
signed int	4bytes	-2147483648 to 2147483647
short int	2bytes	-32768 to 32767
unsigned short int	Range	0 to 65,535
signed short int	Range	-32768 to 32767
long int	4bytes	-2,147,483,647 to 2,147,483,647
signed long int	4bytes	same as long int
unsigned long int	4bytes	0 to 4,294,967,295
float	4bytes	+/- 3.4e +/- 38 (~7 digits)
double	8bytes	+/- 1.7e +/- 308 (~15 digits)
long double	8bytes	+/- 1.7e +/- 308 (~15 digits)
wchar_t	2 or 4 bytes	1 wide character

- The sizes of variables might be different from those shown in the above table, depending on the compiler and the computer you are using.

Example:

```
include <#iostream> using
namespace std;
int main ()
{ std::cout<< "Size of char : " <<sizeof(char) << "\n";
std::cout<< "Size of int : " <<sizeof(int) << "\n";
std::cout<< "Size of short int : " <<sizeof(short int) <<
"\n"; std::cout<< "Size of long int : " <<sizeof(long int) <<
"\n"; std::cout<< "Size of float : " <<sizeof(float) << "\n";
std::cout<< "Size of double : " <<sizeof(double) << "\n";
std::cout<< "Size of wchar_t : " <<sizeof(wchar_t) << "\n";
return 0;}
```

- This example uses “\n”, which inserts a new-line character after every line and << operator is being used to pass multiple values out to the screen. We are also using **sizeof ()** function to get size of various data types.
- When the above code is compiled and executed, it produces the following result which can vary from machine to machine:

```
Size of char : 1
Size of int : 4
Size of short int : 2
Size of long int : 4
Size of float : 4
Size of double : 8
Size of wchar_t : 4
```

- **Arithmetic operators:**

Arithmetic operators are commonly used in a variety of programming languages. In C, there are five of them, and they all take two OPERANDS. Recall that an operand is an expression that is required for an operator to work. For example, for 8 + 4, 8 and 4 are considered as the operands.

Operator Name	Symbol
Multiplication	*
Division	/
Modulus	%
Addition	+
Subtraction	-

Multiplication, addition and subtraction are the simplest to use. Division is also easy, but watch out for the truncation of an int divided by an int! Now, the one that confuses novices is the modulus operator, sometimes known as the remainder operator.

To keep things simple, a%b returns the **REMAINDER** that occurs after performing a/b. For this operator, a and b MUST be integers!

For example, 6%3 returns 0 because 3goes into 6 EXACTLY. Similarly, 4%4, 8%2 and 16%8 all return 0.

So why does 3%4 return 3? Picture it this way: you have 3 holes to fill, but you can only fill 4 holes at a time. You can't fill a group of 4 holes, therefore the 3 holes you had are still empty. Similar story for 7%4 because you can fill in one group of 4 but still have 3 holes remaining.

Example:

```
#include <iostream>
using namespace std;

int main()
{
 int num1 = 10;
 int num2 = 5;

  cout << "Addition = " << num1 + num2 << endl;    cout <<
"Subtraction = " << num1 - num2 << endl;    cout <<
"Multiplication = " << num1 * num2 << endl;
   cout << "Division = " << num1 / num2 << endl;

  return 0;
}
```

Output

Addition=15
Subtraction=5
Multiplication=50
Division = 2

Lab Task 1

Declare different types of variables and find their memory size, location, and values.

```
#include <iostream>   // tells the compiler to include input output stream library
using namespace std;  // Use standard namespace instead of writing std:: for each cin and cout

int main() {                // starting point of the main function
   int age = 20;            // declare integer variable and assign value 20 to it
   double price = 19.99;     // declare double variable and assign value 19.99
   char grade = 'A';         // declare character variable and assign value 'A'
   bool isPassed = true;     // declare boolean variable and assign value true
   string text = "C++";      // declare string variable and assign value "C++"

   cout << "Size of int: " << sizeof(age) << " bytes" << endl;      // Print size of int
   cout << "Size of double: " << sizeof(price) << " bytes" << endl;  // Print size of double
   cout << "Size of char: " << sizeof(grade) << " bytes" << endl;    // Print size of char
   cout << "Size of bool: " << sizeof(isPassed) << " bytes" << endl; // Print size of bool
   cout << "Size of string: " << sizeof(text) << " bytes" << endl;   // Print size of string

   cout << "Value of int: " << age << endl;      // Print value of int
   cout << "Value of double: " << price << endl;  // Print value of double
   cout << "Value of char: " << grade << endl;    // Print value of char
   cout << "Value of bool: " << isPassed << endl; // Print value of bool
   cout << "Value of string: " << text << endl;   // Print value of string

   return 0;  // tells the computer that program has ended successfully
}
```

Explanation

- Different variables (**int, float, char, double**) are declared.
- **sizeof()** is used to find the memory size.
- **& operator** is used to display the memory location (address) of variables.
- **cout** displays the values on the screen.

Lab Task 2

Declare and display a floating-point variable called “pi”.

```
#include <iostream>   // tells the compiler to include input output stream library
using namespace std;  // allows us to use standard namespace without having to write std:: for each cin and cout

int main() {                // starting point of main function from where programs execution begins
   float pi = 3.14;   // Declare floating-point variable pi and assign value 3.14
   cout << "The value of pi is: " << pi << endl;  // Print the value of pi
   return 0;  // tells the computer that program has ended successfully
}
```

Explanation

- A floating-point variable **pi** is declared.
- The value **3.14** is assigned to it.
- The value is displayed using **cout**.

Lab Task 3

Convert temperature from Celsius to Fahrenheit.

Formula

F=(C×9/5)+32F = (C \times 9/5) + 32F=(C×9/5)+32

```
#include <iostream>   // tells the compiler to include input output stream library
using namespace std;  // Use standard namespace instead of writing std:: for each cout and cin

int main() {          // start of main function
   double celsius;   // Declare variable to store Celsius temperature
   cout << "Enter temperature in Celsius: ";  // Prompt user for input
   cin >> celsius;   // takes value from user and stores that value in a variable

   double fahrenheit = (celsius * 9.0 / 5.0) + 32; // Convert Celsius to Fahrenheit using formula
   cout << "Temperature in Fahrenheit: " << fahrenheit << endl; // Print result to display

   return 0;  //tells the computer that program has ended successfully
}
```

Explanation

- The program takes **Celsius temperature as input** from the user.
- It converts the value using the **formula**.
- The result is displayed using **cout**.